

Performance (Desempeño)

Performance = Tiempo + Espacio



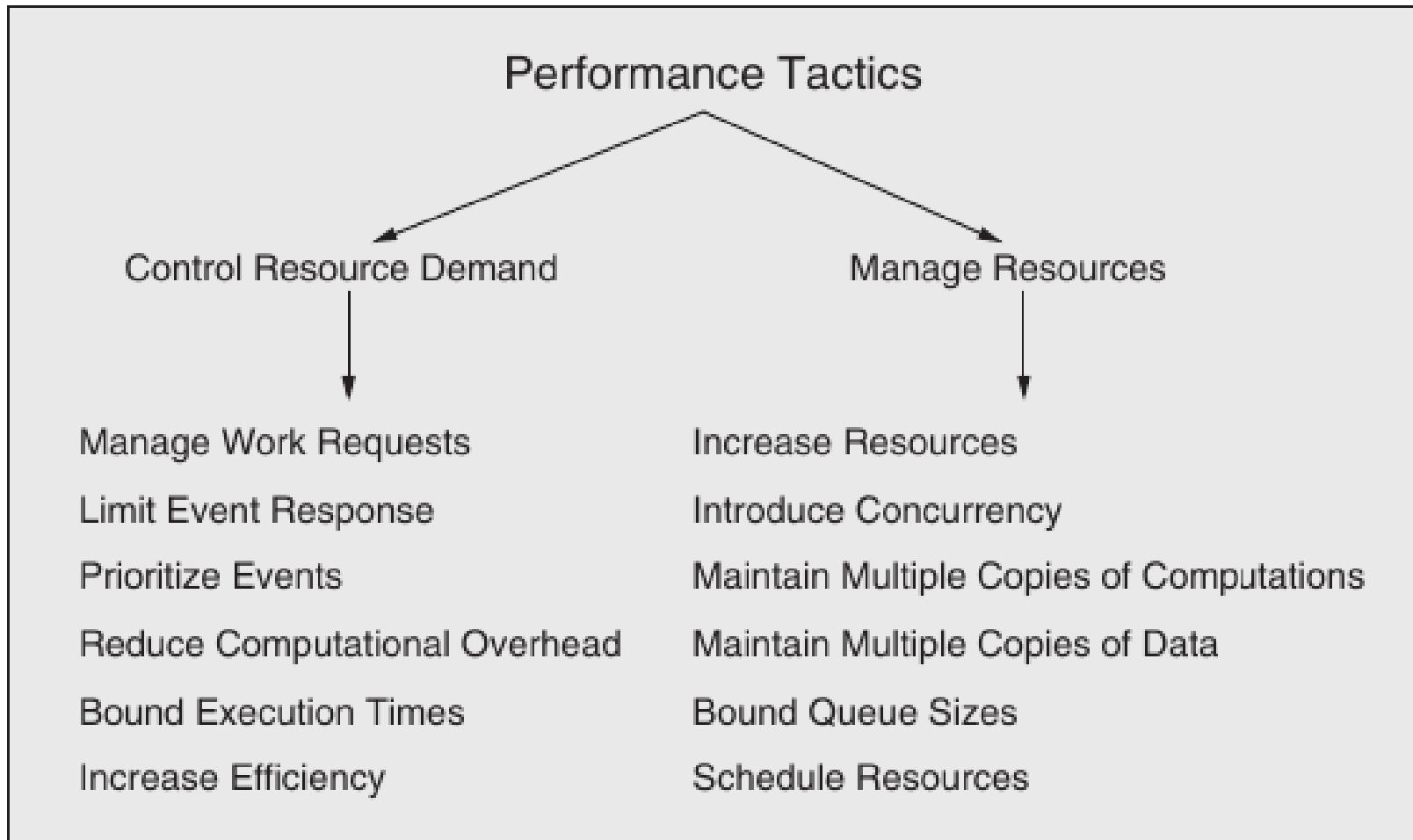
Principales problemas de Desempeño

Principales problemas de Desempeño

El mundo real tiene límites

- Procesos requieren tiempo > 0 para completarse
- Recursos (memoria, disco) son limitados
- Redes tienen ancho de banda y latencia
- Procesos dependen de otros elementos
 - Disponibilidad de recursos (ej. Servicios, Hardware, etc.)
 - Recursos habilitados o deshabilitados
 - Recursos de uso compartido bloqueados por otros procesos
 - Dependencia hacia otros procesos

Tacticas para Desempeño



Controlar demanda de recursos

- ¿Cómo manejar la recepción de eventos en el sistema?
- ¿Cómo procesar los eventos una vez ingresan al sistema?

¿Cómo manejar la recepción de eventos en el sistema?

Administrar peticiones de trabajo

- Gestionar (administrativamente) la llegada de eventos
 - Service Level Agreement (SLA)
 - Procesar N cantidad de eventos por unidad de tiempo
 - Tiempo de respuesta máximo de T



...



- Ajustar tasa de muestreo o procesamiento
 - Tasa de recepción de datos de un sensor
 - Frames per second
 - Respuesta a peticiones por segundo

Limitar respuesta a eventos

- ¿Qué hacer cuando muchos eventos llegan al sistema?
 - a. Encolarlos hasta que puedan ser procesados



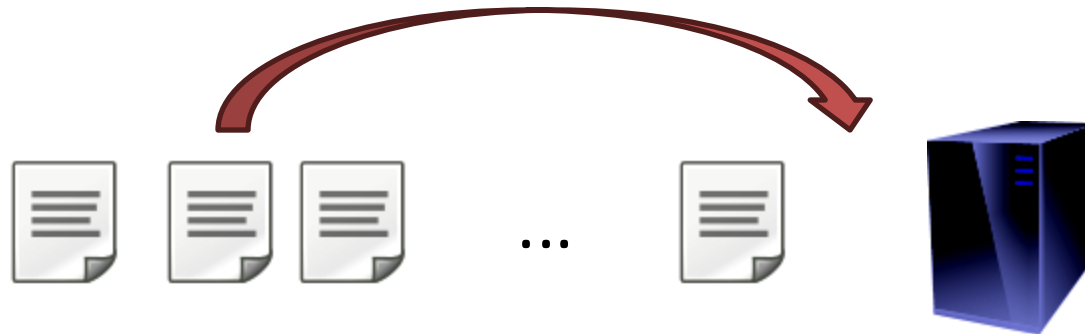
- b. Descartarlos si pasan de cierto límite



- ¿Qué tan crítico es procesar **todos** los eventos?
- ¿Qué hacer cuando se descarta un evento?
 - a. Nada
 - b. Logging
 - c. Notificar a algún usuario

Priorizar eventos

- Procesar primero los eventos más prioritarios
- Postergar (o descartar) eventos menos importantes



¿Cómo procesar los eventos una vez llegan al sistema?

Aumentar eficiencia

- Todo lo que Ud. aprendió en el curso de **Análisis de Algoritmos y Bases de Datos!**
 - Código
 - Buscar algoritmos más eficientes
 - Revisar código con muchos ciclos
 - Usar Map, Set, donde sea necesario
 - Profiling
 - Bases de datos
 - Usar índices en BD
 - Denormalizar donde la velocidad sea crítica

Reducir overhead computacional

- Reducir indirección (reducir intermediarios)
 - [Hole Punching](#) (P2P, VoIP, VPN)
 - C++ inlining, devirtualization
 - Linked lists --> arrays
- Co-ubicar recursos que se comunican intensivamente
 - Ubicarlos en la misma red
 - Ubicarlos en el mismo nodo computacional
- Limpieza periódica. Ejemplos:
 - [Postgres VACUUM](#)
 - Reboot

Limitar tiempos de ejecución

- Limitar cuánto tiempo de procesador se usa en un proceso.
 - Ejemplos (Linux)
 - [cpulimit](#)
 - [cfs](#)

Administrar recursos

Aumentar recursos

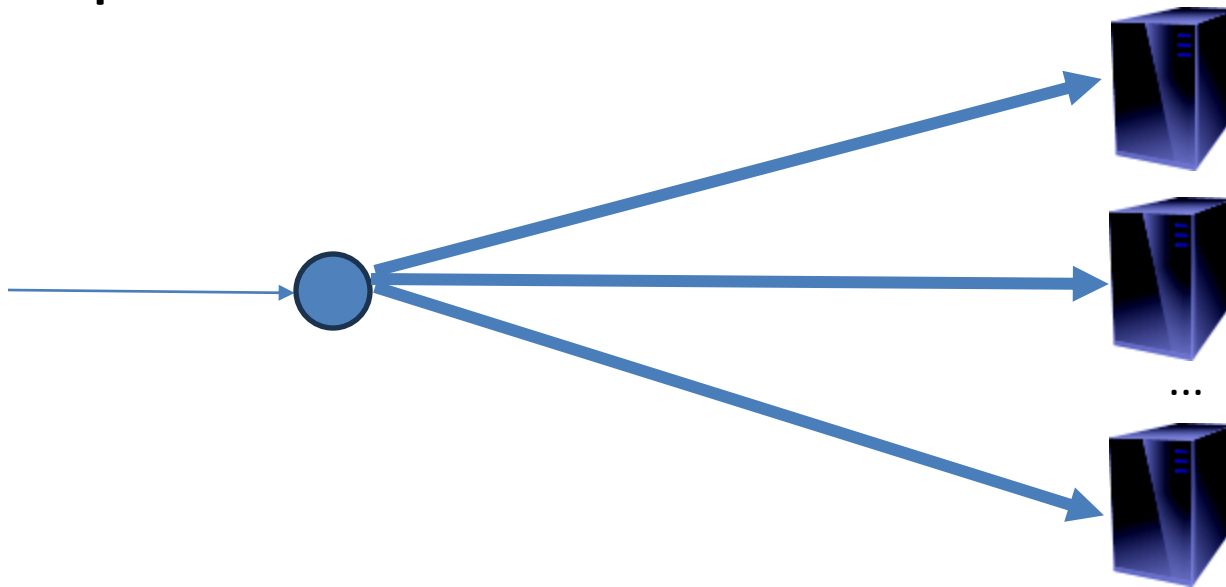
- Aumentar cantidad y capacidad de:
 - CPU
 - Memoria
- Mejorar conexión de red

Paralelismo

- Procesar peticiones en paralelo en un mismo nodo computacional
 - ¿Qué procesos son paralelizables?
 - ¿Tienen acceso a recursos compartidos?

Múltiples nodos computacionales

- Réplicas
- Delegar peticiones a los nodos que estén más disponibles



Copias múltiples de datos

- Replicación de datos
 - Copias de los datos en múltiples nodos similares
 - Replicación de bases de datos
 - Sincronización
- Caching
 - Copia de los datos (o parte de ellos) en un medio de almacenamiento más rápido
 - Memoria RAM vs disco
 - Local vs remoto
 - Ejemplos
 - Redis
 - Memcached
 - [Spring Boot Cache](#)
 - CDN
 - Cache del navegador
 - CPU L1/L2/L3 cache

Limitar tamaño de colas

- Relacionado con táctica "Limitar respuesta a eventos"
 - Tamaño máximo de cola
 - ¿Qué hacer cuando la cola se llena?
 - Descartar y logging
 - Aumentar tamaño cola

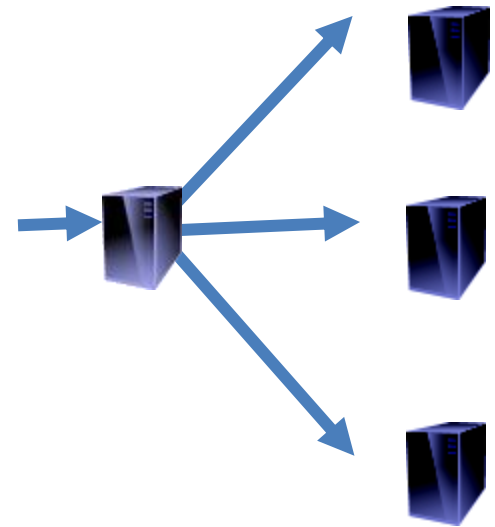
Programar recursos (Schedule resources)

- Determinar cuándo se utilizará un recurso
 - ¿Qué tan urgente es responder a una petición?
- Ejemplos
 - Scheduling de procesadores (Kernel de Sistemas Operativos)
 - Scheduling de paquetes en la red. [Traffic shaping](#)
 - Buffers de escritura de archivos (Ej. USB drives)

Patrones de Desempeño

Balanceador de Carga (Load Balancer)

- Redirige las peticiones dependiendo de la disponibilidad de cada réplica
 - Táctica: Schedule resources
- Ventajas
 - Encapsula fallos en réplicas
 - Reduce latencia
 - Facilita escalabilidad
- Desventajas
 - Punto único de fallo
 - Potencial cuello de botella

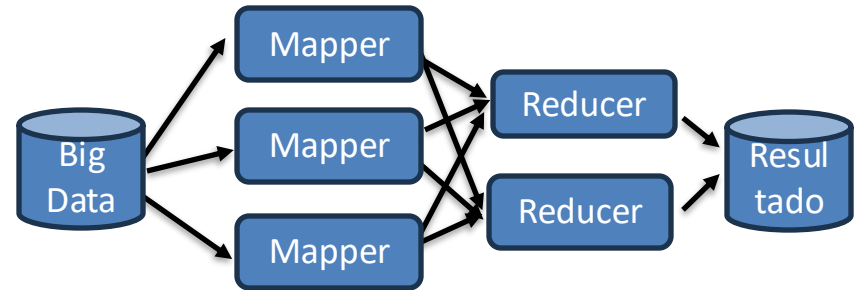


Throttling

- Limitar la cantidad de peticiones que recibe un servicio
 - Tácticas: Administrar peticiones de trabajo, CircuitBreaker
- Ejemplos: Spring Cloud Gateway, Bucket4j, Nginx, Throttling de mensajes en Kafka
- Ventaja
 - Reduce probabilidad de sobrecarga
- Desventajas
 - Potencial cuello de botella
 - Si demanda es muy grande, se pueden perder peticiones
 - Dificultad de agregar en sistemas existentes

Map-Reduce

- Procesamiento paralelo de datos
- Map
 - Entradas:
 - Lista o stream de datos
 - Función (Mapper)
 - Salida: Lista o stream de datos resultantes de aplicar la función a los datos de entrada
- Reduce
 - Entradas
 - Lista o stream de datos
 - Función de consolidación (Reducer)
 - Salida: Datos consolidados resultantes de aplicar la función a los datos de entrada



- Ventajas
 - Eficiencia para grandes datos
 - Fallos afectan solo una parte del procesamiento
- Desventajas
 - Overhead podría no justificarse para datos pequeños
 - No sirve si no se puede paralelizar el procesamiento de datos